

Warping the Edge: Enabling Instant Mobility for Stateful Applications over 5G and Beyond

Mukhtiar Ahmad^{†¶}, Faaq Bilal[†], Mutahar Ali[†], Syed Muhammad Ali Nawazish[†],
Amir Salman[‡], Shazer Ali[†], Fawad Ahmad[§], Zafar Ayyub Qazi^{†¶}

[†]Lahore University of Management Sciences (LUMS), Lahore, Pakistan

[¶]King Abdullah University of Science and Technology (KAUST), Thuwal, Saudi Arabia

[‡]Independent Researcher, Lahore, Pakistan

[§]Rochester Institute of Technology (RIT), Rochester, NY, USA

Abstract

Real-time mobile applications such as AR/VR, cloud gaming, and collaborative robotics rely on edge computing to maintain ultra-low latency, yet they can suffer noticeable service interruptions when users move because the application's session state must migrate to a new edge site. We present EdgeWarp, a system that delivers instant mobility for stateful edge applications over 5G while laying the groundwork for 6G. EdgeWarp employs a two-step synchronization protocol that proactively mirrors application state at edge sites predicted to serve the user next, sharply reducing transfer delays, and it signals each session's latency budget to the 5G control plane so that critical flows are prioritized during handover. Experiments with real applications and 4G/5G radio traces show that EdgeWarp cuts mobility-induced downtime by up to 15.4×, charting a path toward zero-downtime edge computing—a prerequisite for emerging 6G scenarios such as holographic telepresence, tactile Internet, and large-scale digital twins. We have made our anonymized code publicly accessible [here](#).

CCS Concepts

• **Networks** → **Mobile networks; Network mobility; Cross-layer protocols.**

ACM Reference Format:

Mukhtiar Ahmad^{†¶}, Faaq Bilal[†], Mutahar Ali[†], Syed Muhammad Ali Nawazish[†], Amir Salman[‡], Shazer Ali[†], Fawad Ahmad[§], Zafar Ayyub Qazi^{†¶}. 2025. Warping the Edge: Enabling Instant Mobility for Stateful Applications over 5G and Beyond. In *The Tenth ACM/IEEE Symposium on Edge Computing (SEC '25)*, December 3–6, 2025, Arlington, VA, USA. ACM, New York, NY, USA, 18 pages. <https://doi.org/10.1145/3769102.3770610>



This work is licensed under a Creative Commons Attribution 4.0 International License.

SEC '25, Arlington, VA, USA

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2238-7/2025/12

<https://doi.org/10.1145/3769102.3770610>

1 Introduction

5G and beyond networks aim to support intriguing emerging latency-sensitive applications (apps) that promise to enhance safety and user experience. These apps encompass a range of use cases, such as connected and autonomous vehicles [11, 51, 112], virtual and augmented reality (AR/VR) [35, 90], edge-based video analytics [18], and multi-player online gaming [91, 111]. Such apps have stringent delay budgets, e.g., VR apps with head tracking systems require latency of less than 17 ms to achieve perceptual stability [35, 90]. To enable support for such apps, 5G networks are integrating Edge Computing [46, 90]; a paradigm whereby apps are hosted closer to the users, on cell towers, cell aggregation sites, or other edge sites [44–46]. Many cellular providers are partnering with cloud providers for deploying edge apps inside the cellular infrastructure [17, 33, 77, 92].

Hosting apps at the network edge can significantly reduce delays. However, it also makes it challenging to ensure that users always receive low latency as they move across different locations. This, in turn, requires support to handover clients to different edge sites with negligible delays [43]. These app handovers are likely to be frequent [9, 41, 44, 46, 70, 114] for a significant fraction of 5G use cases as more than 50% of 5G IoTs are expected to be highly mobile apps related to the automotive industry [20].¹ Many of these edge apps are stateful [11, 56, 95, 111, 112] and migrating stateful apps over 5G can cause significant app downtime during mobility [10, 108] (§2.4). Consequently, if the app downtime exceeds the delay budget for latency-sensitive apps, it can compromise user safety in the case of connected cars [11, 112] or cause users motion sickness when using immersive mobile VR [58].

In this paper, we first step back and ask the question, *what are the key architectural bottlenecks in enabling stateful mobile edge apps over 5G?* We identify three key issues in enabling fast app handovers during mobility (illustrated in Figure 1):

¹5G networks will have over 7 billion connected IoTs by 2030 [37].

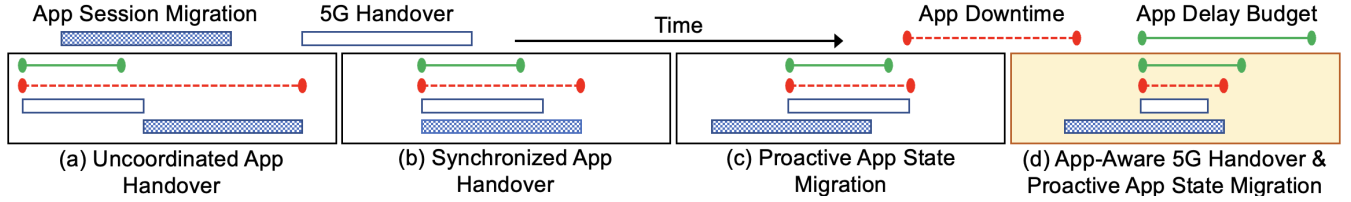


Figure 1: Key challenges and potential solutions to minimize downtime during application handover caused by mobility. Typical 5G handover delays are in the order of 100 to 200 ms [106], while application session migration delays exceed 140 ms with existing methods (§3.1).

- **Uncoordinated and Reactive App Session Migration:** Stateful app handover involves two key steps: (i) the user’s network session migration, also known as 5G handover² and (ii) the user’s app session migration. Uncoordinated migration of the network and app sessions can cause extended app downtime (Figure 1a). If the app session migration occurs after a 5G handover, it can result in two interruptions, one during the 5G handover and another during the app session migration. Even if app session migration is performed proactively, latency-sensitive apps that frequently update app state [111] can incur non-zero downtime to retrieve the latest app state. On the other hand, even if we perfectly synchronize 5G handover and app session migration events (Figure 1b), app downtime can still exceed the delay budget (< 17 ms [35, 90]) of latency-sensitive edge apps.
- **Slow App Session Migration:** A key challenge with speeding up session migration of stateful apps is that they typically require the user’s up-to-date app state at the target edge host before a user connects to it [49]. However, existing techniques for app session migration are either slow [10, 31, 32, 72, 73, 113] or rely on app-specific optimizations [26] (§3.1). Moreover, simpler session migration techniques, such as continuing the app session with the source instance before state migration completes, can result in two interruptions: one during the 5G handover and another during app session migration. This occurs because the app session at the source edge host needs to be frozen, migrated, and restarted on the target edge host to ensure state consistency. The key challenge is to design a session migration technique that is *fast* and *generic*.
- **App-agnostic 5G Handover:** We observe that even if we somehow address the issue of slow app session migration, the time required for a 5G handover can become a bottleneck in enabling latency-sensitive edge apps, as shown in Figure 1c (§3.3). An architectural issue is that 5G control plane processing is *application-agnostic* and there are no 5G interfaces through which edge apps can inform the cellular control plane of their latency requirements.

To address the above issues, we propose a new system design, *EdgeWarp*, for edge apps over 5G. *EdgeWarp* is a cross-layer design that consists of the following key ideas:

- **Target BS Prediction:** A coordinated and proactive approach to app handovers is essential to minimize app downtimes. To achieve this, *EdgeWarp* predicts the target BS³ for a 5G handover in advance, allowing edge apps to initiate app state migration proactively (Figure 1c). Existing techniques for target BS prediction rely on handover history [8, 22], LTE events [98], or machine learning approaches [29, 60, 86]. Among these, the 5G network-based approaches have two common limitations: (i) they lack early prediction timing, which is crucial for practical benefits, as predicting the target BS only a few milliseconds before a handover provides negligible performance gains (§6.5.1), and (ii) their prediction accuracy is generally poor ($< 85\%$).

To overcome these limitations, we propose a multi-step early target BS prediction pipeline (§4.3), which leverages Radio Network Information (RNI) APIs in MEC [42] to predict handovers in advance using an LSTM network, then extrapolate radio measurements to enable early target BS prediction, and finally perform target BS prediction. We are using LSTMs because they excel at capturing both short-term fluctuations and long-term patterns in the rapidly changing time series data from 5G mmWave radio access networks [16]. We evaluate this pipeline on real 5G/4G radio network traces, including data from driving tests [27], high-speed trains [64], and various other mobility scenarios [79]. Our evaluation shows that on average, 97% correct predictions of the target BS 100 ms ahead of the handover (§6.3), which we refer to as a *mobility hint* for apps.

The novelty of our approach lies in the synthesis of this multi-step pipeline with radio signal extrapolation. This unique combination enables predictions 100 ms in advance with 97% average accuracy, a significant improvement over prior work that achieves $< 85\%$ accuracy [86] only a few

²A 5G handover can also trigger user plane session migration—referred to as Packet Data Unit session migration [3, 93].

³Predicting the target BS is equivalent to predicting the target edge host as it is a function of target BS. Details in §4.1.

milliseconds before the event. Our performance improvement stems from leveraging (i) enhanced feature vectors with an LSTM network that capture time-series radio signal trends and (ii) the extrapolation of radio signals (§4.3) when sample data is insufficient.

- **Two-Step App State Synchronization:** An app can proactively synchronize all or a subset of the user’s app state with the target edge host based on a mobility hint. However, synchronizing a highly dynamic state object that is frequently updated offers little advantage as it quickly becomes stale. Additionally, identifying and tracking different types of state objects for proactive synchronization is a daunting task for developers.

To address this, we propose extending existing edge data stores (e.g., Redis [87] and Memcached [76]) to automatically track the update rate for each state object of an edge app (§4.4). The novelty of our approach lies not in merely using an external store, but in the two-step migration protocol this enables. Our modified data store exposes new APIs (Table 2) to first, proactively synchronize only the state objects with low update rates upon receiving a mobility hint, which minimizes bandwidth overhead. Second, the remaining, more dynamic app state is synchronized during the 5G handover itself to maximally overlap with the interruption time (Figure 1c). This approach reduces blocking time during state migration, resulting in minimized app downtime (§6.4).

- **App-Aware Control Plane Processing:** To make 5G control plane processing app-aware, we introduce new APIs (Table 3) that allow edge apps to report their latency requirements and prioritize control plane processing accordingly (§4.5). These APIs are called by the server side of latency-sensitive apps through the MEC [46], which influences control plane processing. Restricting API access to the server side ensures better control and prevents misuse, as it is under the network operator’s control or managed by trusted third parties. Our experiments demonstrate that an app-aware 5G control plane significantly reduces handover delays, bringing edge app downtime within their specified delay budgets (§6.6).

To show the efficacy of the above ideas, we implement *EdgeWarp*. We implement a multi-step target BS prediction pipeline utilizing an 8-layer stacked LSTM neural network on top of Keras library [57]. Our data sets consist of real 4G/5G traces with a total of 23630 handovers. On the app layer, we implement dynamic state tracking and two-step state synchronization in Redis [87] and Memcached [76] data stores (§4.4). To evaluate the impact of *EdgeWarp* on real edge apps, we modify two recent open-source edge apps, CarMap [11] and EMP [112], to use our modified Redis data

store for proactive state synchronization. As real apps provide limited options to vary app state properties (e.g., state sizes, update rate, etc.), we implement a simulation framework in Python to simulate the behavior of a wide variety of latency-sensitive edge apps (§5.2). We interface a Python version of both modified Memcached and Redis key-value data stores with the simulation framework to evaluate the impact of two-step state syncing on a wide variety of app behaviors. To implement app-aware control plane processing, we modify a recently proposed cellular control plane [12]. We provide further details of our implementations in §5. We open-source our code on an anonymous Github link: <https://github.com/stonegen/SEC-EdgeWarp>.

Our evaluation shows up to 14× and 15.4× improvement in app downtime for CarMap and EMP, respectively. We correctly predict the target BS in 97% cases at least 100 ms in advance, in 1.3% cases less than 100 ms in advance, and fail to predict the correct target BS in 1.7% cases. Apps can significantly improve performance during mobility with correct target BS predictions (Figure §1c), while neither improving nor degrading performance in the case of false predictions due to fallback on reactive migration (Figure §1b). Consequently, both CarMap and EMP show more than 2.5× improvement in app state migration time (§6.4). *EdgeWarp* shows up to $10^3\times$ improvement in median 5G handover completion time for latency-sensitive apps (§6.6).

Contributions: In summary, we make the following contributions to this work:

- (1) We identify the key architectural bottlenecks in enabling stateful mobile apps over 5G (§3).
- (2) We design a new system architecture, *EdgeWarp*, compatible with existing RAN and cellular core (e.g., 5G/4G/LTE). *EdgeWarp* consists of three new modules to speed up app handovers (§4).
- (3) We implement *EdgeWarp*. Our implementation consists of a target BS predictor, an extended edge data store on top of Redis and Memcached, and an app-aware 5G control plane (§5).
- (4) We evaluate *EdgeWarp* with two real edge apps, CarMap, and EMP, with multiple real-world cellular data sets, and also perform a sensitivity analysis using a simulation framework (§6).

2 Background and Motivation

2.1 5G Architecture

A 5G network consists of User Equipment (UE), Radio Access Network (RAN), and a 5G Core (5GC), as shown in Figure 2. 5GC is decomposed into virtualized microservices in the service-based architecture introduced by 5G. The service-based 5G architecture also enables the concept of network slices [19, 28, 38]. A 5G network can be configured to produce

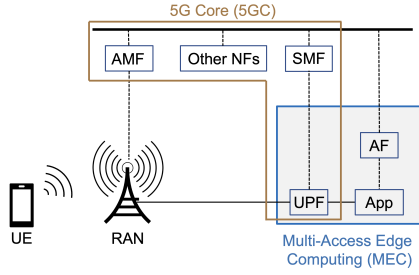


Figure 2: Simplified 5G network architecture.

different logical networks (network slices) on top of shared physical infrastructure for different classes of app traffic. Below we describe the key 5G core functions.

Data Plane Functions: The User Plane Function (UPF) performs the key data plane functions such as connecting base stations to external data networks (DN), IP address allocation, traffic routing, usage reporting, and buffering of downlink traffic. One or more UPFs can serve a single user.

Mobility and Session Management Functions: The Access and Mobility Management Function (AMF) performs the key access and mobility-related functions, including access authentication, access authorization, and mobility management. Every UE is served by a single AMF. The Session Management Function (SMF) performs session creation, modification and release, UPF selection, and downlink data notification. One or more SMFs can serve a single UE.

2.2 Edge Computing Support in 5G

Multi-access Edge Computing (MEC): MEC is a system architecture aimed to provide storage and processing to mobile users at the network edge for low latency and bandwidth efficiency [46]. The 5G system architecture [6] allows the MEC system to collaboratively interact with the 5G system in traffic routing and policy control. For instance, the MEC system can influence the selection or re-selection of an appropriate UPF to route data traffic to a suitable edge app. The 5G network can also expose real-time radio network information to apps through the MEC.

Application Function: MEC allows apps through the Application Function (AF) [6] to route data traffic to the targeted apps at the edge sites. Since a UPF plays a central role in traffic routing, an edge app through an AF can influence the 5G core network to select a particular UPF for the UE [6]. An AF can also influence 5G core network policies and expose services to the users.

2.3 Stateful Edge Applications

5G networks aim to support a large number of edge apps with MEC [44]. These apps can be either stateless or stateful. By stateful apps we mean apps whose processing depends on the state previously reported by the client and/or generated

at the server. An example of a simple stateful app is an edge server maintaining a count of the number of packets received from a client [49]. On user mobility, the user app state (e.g., packets count) will be required at the new edge server (to maintain a correct packets count) before a user connects to it. Real-life stateful edge apps can be much more complex. Below, we discuss a few of these stateful edge apps that are latency-sensitive and can be mobile.

Real-Time Mapping and Localization: Aerial and ground robots (e.g., CarMap [47]) build real-time 3D maps of the environment and use these maps to accurately position themselves with respect to their surroundings [23, 34, 47, 105]. These 3D maps are updated using onboard depth perception sensors, such as stereo cameras and LiDARs. These maps are typically maintained at the network edge for real-time localization in a dynamic environment [107].

Autonomous Vehicles: Autonomous vehicles can leverage edge computing and sensing capabilities, such as infrastructure-mounted LiDARs and stereo cameras, for safer and more efficient driving [24, 101, 112]. EMP [112] is an example of edge-assisted multi-vehicle perception system for connected and autonomous vehicles (CAVs). Such systems merge the raw sensor data shared by multiple CAVs on an edge server to enhance the perception of participating CAVs to improve road safety and traffic efficiency.

Mobile VR: Apps such as those based on augmented reality (AR) and virtual reality (VR) require ultra-low latency in the order of a few milliseconds [85] to provide perceptual stability. Mobile devices running VR Apps can offload computation to MEC for efficiency [44]. Edge-based VR services [53, 56, 95] are examples of such Apps.

Online Gaming: Massive multiplayer online gaming is another example of stateful services at the network edge to achieve ultra-low-latency [21, 68, 111]. Such apps can have highly dynamic app state (i.e., > 100 updates/s) and latency budget < 20 ms.

2.4 Slow Application Handover

App handover for latency-sensitive edge apps should be performed with minimal delay to reduce app downtime.⁴ The time taken for an app handover depends on many factors, including user-specific app state size, state update frequency, app code size, and available network bandwidth.

Figure 3 shows the app downtime during an app handover (due to mobility) for three edge apps having different user-specific dynamic state sizes.⁵ The Figure shows that app downtime for latency-sensitive apps can be up to 9× of their

⁴The duration for which an app is unable to serve the user.

⁵These results are obtained by interfacing the CarMap and EMP (§5.2) with *EdgeWarp* and using our emulation framework (§5.3) for Mobile VR. We assume these apps require strict consistency [100].

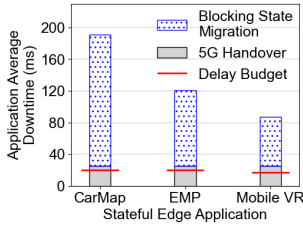


Figure 3: App downtime experienced by latency-sensitive edge apps during mobility.

delay budget. Secondly, for all three apps, the downtime is larger than the delay budget for the respective apps. An app downtime exceeding the delay budget can degrade app performance, such as compromise user safety in the case of connected cars [11, 112] or cause users motion sickness when using immersive mobile VR [58].

2.5 Frequent Application Handover

Prior studies suggest app handovers are likely to be frequent [9, 41, 44, 46, 70, 114]. A recent 5G measurement study [82] shows 30+ 5G handovers can occur in less than eight minutes duration. Similarly, another study shows that on average, 4G handovers can occur every 70 seconds in walking scenarios [66]. When edge apps are deployed beside base stations, every network handover may trigger an app session migration. Even if edge apps are hosted at BS aggregation points, an app session migration may happen frequently for mobile users [15]. These app migrations can result in frequent app downtime, as discussed in §2.4.

3 Issues and Challenges

In this section, we identify the key issues and challenges in enabling stateful mobile edge apps. We also highlight the impact of these issues through testbed experiments and qualitative discussion.

3.1 Slow Application Session Migration

Most of the existing app session migration techniques are based on containers [10, 72, 73, 83, 113], VMs [71], and user-space tools [32, 52]. Table 1 shows CarMap [11] app downtime during an app handover when using these session migration techniques. Table 1 shows that container-based migration provides an app downtime >1.2 s which is significantly higher than the delay budget [15] of latency-sensitive edge apps (e.g., CarMap [11, 15] has a network delay budget of ≈ 20 ms within its total delay budget of 100 ms, of which ≈ 80 ms is consumed by the processing stack). VM-based migration is even slower. A widely used user-space tool CRIU [32] checkpoints a running app as a collection

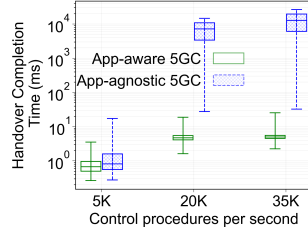


Figure 4: Handover completion time comparison when apps are served by different 5G cores.

App HO Method	App downtime	App agnostic?	Meet App delay budget?
VM	> 4 s	Yes	No
Container	> 1.2 s	Yes	No
CRIU	> 140 ms	Yes	No

Table 1: Downtime for CarMap (network delay budget [15] ≈ 20 ms) during an App handover (HO) with userspace tool CRIU [32], Container [10, 48, 73, 83] (Docker and LXC), and VMs [73].

of files and resumes it on the destination machine. Table 1 shows that CRIU provides an app downtime >140 ms, which is still above the delay budget of latency-sensitive apps. On the other hand, app-specific optimizations-based techniques are relatively fast [26], however, they require the app developer to identify the user’s dynamic app state and the checkpointing occasions (preferably idle times) to avoid or reduce performance degradation [102, 103].

Challenge: The generic session migration techniques, such as VM or container-based migration are slow. User space migration techniques, such as CRIU, are relatively fast but still exceed the delay budgets of many apps. Whereas app-specific techniques are not developer-friendly due to the development efforts involved in optimizing every app. We look at this problem from a different lens, asking the question: *can the 5G network help expedite app session migration under mobility while requiring minimal app-specific changes from the developer?*

3.2 Reactive Application Session Migration

When performing reactive app handover, the minimum app downtime is the maximum delay of app session migration and 5G handover (Figure 1b). On the other hand, proactive session migration requires information about the target BS before a 5G handover starts. Our analysis of the 4G/5G radio traces, having 23630 handover events shows on average 7.45 candidate BSs for a handover (range between 1-20). The latency-sensitive apps we explored have dynamic state sizes of ≈ 3 MBs (§6.4.1 and §6.4.2) and can have an update rate of 100 updates/s [111]. Proactively synchronizing app state on multiple BSs (on average 7.45) is highly inefficient in terms of bandwidth.

Challenge: Proactive app session migration on multiple BSs is not efficient. As multiple candidate BSs are available for a handover in which only one target BS is selected at the time of handover, *it is challenging to determine the target BS sufficiently in advance and with reasonable accuracy* to enable proactive app session migration.

3.3 Application-Agnostic 5G Handover

To highlight how app-agnostic 5G handover events can impact the downtimes for edge apps, we conduct testbed experiments. Figure 4 shows handover completion time for latency-sensitive apps in different control plane load scenarios; where the load corresponds to UEs serving different applications, such as AR/VR, connected and autonomous vehicles, and haptic feedback [37, 40, 65]. The 5G control plane in Figure 4 is serving a 12:88 ratio (projected IoT workload for 5G [74]) of latency-sensitive and latency-tolerant traffic. We compare two schemes: (i) app-agnostic 5GC represents the existing control plane implementation where apps do not influence the control plane and (ii) app-aware 5GC represents an ideal 5G control plane that is aware of the delay budgets of the underlying apps running on UEs. When the load on the 5G control plane is low (i.e., 5K procedures⁶ per second), app-aware 5GC provides a 1.2× improvement in median handover completion time over the app-agnostic 5GC. As the load increases on the control plane, we observe up to three orders of magnitude improvement in median handover completion times with app-aware 5GC. The load scenarios in Figure 4 may be frequent in 5G as it aims to support massive machine-type communication with a connection density of up to 1 million devices per square km [30].

Challenge: Edge sites are known to have limited resources [25, 99, 104] therefore reserving control plane resources for every application class may not be efficient.⁷ In some cases, a shared control plane for different applications is inevitable when a UE connects to multiple network slices in 5G. A challenge here is to support latency-sensitive apps in an edge-based cellular core deployment with efficient resource utilization. In the existing 3GPP specifications [5–7], the cellular control plane cannot differentiate between different application classes and it does not provide any interface/protocol for AF to influence the processing of the control plane.

4 Design

To address the issues in §3, we design a new system architecture, *EdgeWarp*. *EdgeWarp* is a cross-layer design that (a) exposes radio network information (reported by the UE to the BS) through the MEC APIs [42] to edge apps so they can proactively perform application state migration, and (b) shares application latency requirements with the 5G control plane for speeding 5G handovers. *EdgeWarp* consists of three new modules: (i) a module for proactively predicting target BSs for mobile clients (§4.3), (ii) an edge data store

for stateful edge apps that can leverage proactive network information to speed up application state synchronization (§4.4), and (iii) app-aware control plane for speeding up 5G handover events (§4.5). In this section, we first discuss our goals and assumptions in designing *EdgeWarp* (§4.1), then provide an overview of our proposed architecture (§4.2), before discussing the individual modules in detail.

4.1 Goals and Assumptions

The key goal of *EdgeWarp* is to *reduce app downtime for stateful edge apps when migrating application state due to client mobility*. At the same time, *EdgeWarp* aims to incur minimal latency and bandwidth overheads in non-mobility scenarios. We next make explicit our assumptions about the deployment and computational model before diving into our design.

- We assume edge deployment of the cellular core network which is the preferred deployment option for latency-sensitive apps [37, 38].
- We assume a client-server model for apps. An app server hosted at the network edge provides services to mobile clients. A stateful app server maintains a per-user app state.
- We assume edge apps provide Read Your Writes consistency, which is *the effect of a write operation by a process (user app) on data item x (user's app state) will always be seen by a successive read operation on x by the same process*. A system designed for Read Your Writes consistency can be easily tuned for apps that can tolerate relaxed consistency models [100].
- Our design supports flexible deployment scenarios, including both single and multiple BSs per MEC site. For our evaluation, we focus on a model where each MEC site aggregates multiple BSs, which is supported by our dataset featuring an average of seven candidate BSs per handover. Consequently, an app state migration is only triggered when a handover occurs between BSs served by different edge sites.

4.2 EdgeWarp's System Architecture

Figure 5 provides an overview of *EdgeWarp*'s system architecture. *EdgeWarp* argues to store the user's app state in our modified edge data store while the application logic is stateless. To enable this, the modified data store exposes new APIs (Table 2) to applications. The target BS predictor module receives real-time radio network information (RNI) exposed to applications in 5G [42, 75] and predicts the destination BS in advance before an actual handover starts. The mobility handler module receives information about the target BS from the target BS predictor module and provides the target edge host and application instance information (such as IP address, port number, etc.) to the source app instance

⁶A procedure is a set of related request/response messages between a UE and cellular control plane, i.e., handover procedure.

⁷Network slices such as shown in Figure 5 of the [38] reserves control plane for different application classes which may not be resource efficient in an edge-based cellular core deployment [37].

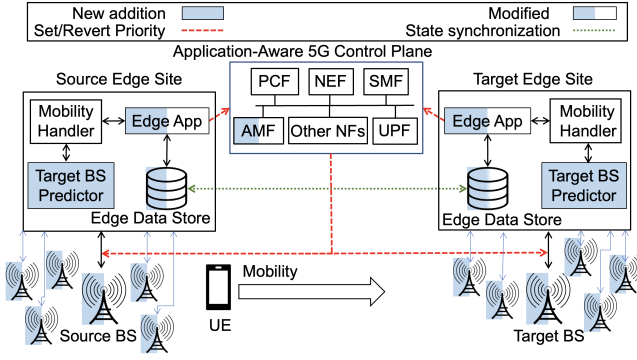


Figure 5: *EdgeWarp*'s system architecture.

to facilitate proactive application state migration. The app-aware cellular control plane leverages the application-level information (such as latency needs) to provide priority processing for latency-sensitive edge apps. To enable this, the app-aware cellular control plane exposes new APIs (Table 3) to edge apps to prioritize their control plane processing.

4.3 Target BS Prediction

EdgeWarp introduces a novel approach to predict target BS well in advance of handover decisions, enabling proactive application state migration. Our multi-step pipeline tackles this as a multi-class classification problem with dynamic classes, where each BS represents a class and the set of neighboring BSs can change over time. The pipeline leverages real-time radio signal strength measurements from Radio Network Information (RNI) APIs [42], utilizing Reference Signal Received Power (RSRP) and Reference Signal Received Quality (RSRQ). For high-accuracy predictions, we employ a stacked Long Short-Term Memory (LSTM) network. We chose LSTM for its ability to handle complex, non-linear mobile network data, capturing both short-term fluctuations and long-term patterns in rapidly changing 5G radio signals.

Our approach achieves 97% prediction accuracy (§6.3) without extensive per-user data storage, outperforming other early target BS prediction schemes (typically <85% accurate). The LSTM model, trained on real 4G/5G network traces from various mobility scenarios, forms the core of our prediction pipeline (Figure 6):

- (1) **LSTM Network Prediction:** We apply an LSTM network (§5.1) to predict handover probabilities from the source BS to each neighboring BS with sufficient signal strength measurements. The network utilizes a feature vector for each BS pair. This vector comprises N absolute values and N relative values (compared to the source BS) of both RSRP and RSRQ measurements. The number N is configurable, with a default value of 6 set based on our evaluation results. This approach allows the LSTM to capture both the absolute signal strengths and their

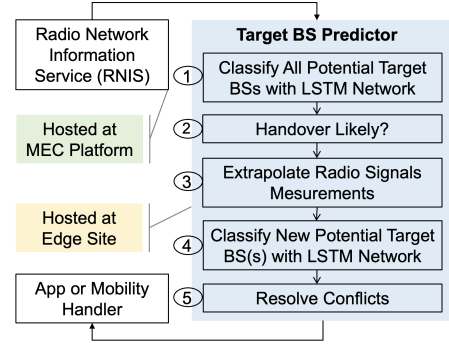


Figure 6: *EdgeWarp*'s target BS prediction.

relative changes, enabling more accurate predictions of potential handovers.

- (2) **Handover Likelihood Assessment:** We flag a high likelihood of handover when the predicted handover probability to any target BS exceeds a learned threshold, let's say P^8 .
- (3) **Early Prediction Handling:** For cases where the actual target BS has fewer than N samples (approximately 5% of cases in our radio traces), we perform zero-order extrapolation of RSRP and RSRQ to complete the required window size.
- (4) **Extrapolated Target BSs Prediction:** We apply the LSTM classifier to new source-target BS pairs as they become available.
- (5) **Conflict Resolution:** We determine the top- x BSs from the pool of candidates based on handover probabilities assigned by the LSTM network. The value of x is configurable (default = 1) to accommodate specific edge application requirements.

4.4 Two-Step App State Synchronization

In this section, we discuss our modifications to the existing data stores to provide two-step state synchronization based on mobility hints.

Modification to Data Store: Our modifications to the data store include keeping an additional meta-state with every key to track whether a state object is updated after the previous synchronization event and sorting state objects based on their update rates. The additional per-key meta-state in the data store at the source edge host includes:

- *First update time:* The time the key was created in the data store.
- *Last update time:* The time when the key was last time updated in the data store.
- *Updates count:* The total number of key updates.

⁸We learned the value of $P = 0.37$ that maximizes the F1-score of the binary LSTM classifier.

- *Last sync time*: The time when the key was last synced with the data store instance at the target edge host. To avoid any conflict when an update and sync operation are performed in parallel, the external data store records the time before the sync process starts for a key (say T) and updates T as the last sync time when the sync operation completes.

The meta-state gives useful information about (i) the average key update rate and (ii) whether the key is updated after the last sync. We leverage the per-key meta-state to define additional APIs (in Table 2) in the modified data store.

Background State Synchronization: An edge app registers itself with the mobility handler module (Figure 5) to receive important network events, such as mobility hint and mobility handover. The mobility handler module receives target BS information from the target BS prediction module and provides destination edge host and app instance information (i.e., IP address, port number) to the source edge app. On reception of a mobility hint, the source edge app calls *BackgroundSync* API exposed by our modified data store. Call to the *BackgroundSync* API initiates the syncing of the user’s app state with the target data store instance by synchronizing the state object that has the lowest update rate and was updated after the last sync event (update time higher than sync time). The background syncing blocks updates to the key only when getting a copy of the state. The migration and restoration in the target data store instance is a non-blocking operation. As read operations are very fast in edge-suitable data stores (such as Redis and Memcached), this allows the app to process users’ requests with minimal overhead during background state syncing.

Blocking State Synchronization: On a 5G handover signal from the mobility handler module, the source edge app calls *BlockingSync* API in Table 2 by providing it with a callback function. This API synchronizes the rest of the state that is either not synced or updated after the last sync. In the best case scenario, all the user’s app state is migrated during the background syncing and the callback function returns right after the call to the *BlockingSync* API. In the worst case, all the user’s app state needs to be synced due to post-sync updates. The duration from call to *BlockingSync* API to the callback call is the blocking state migration time—also the minimum app downtime during an app handover. If the third parameter (retain) in *BlockingSync* API is true, the source data store instance will not delete the user state otherwise it will be deleted after the migration completes.

4.5 App-Aware Control Plane Processing

In this section, we discuss how an AMF can differentiate between different types of UEs. We also describe how an AMF

New APIs in Modified Data Store	
	<i>BackgroundSync (userID, destHost)</i>
	<i>BlockingSync (userID, callBackFunc, retain)</i>

Table 2: New APIs for latency-sensitive edge apps in the modified data store tailored to the needs of latency-sensitive mobile applications.

NF	New APIs in App-Aware 5GC for AF
AMF	<i>Set_Priority (IpAddress, priority)</i>
AMF	<i>Revert_Priority (IpAddress)</i>
RAN	<i>Set_Priority (RAN UE NG-AP ID, priority)</i>
RAN	<i>Revert_Priority (RAN UE NG-AP ID)</i>

Table 3: New APIs available in *EdgeWarp* for an AF to influence control plane operations at the RAN/AMF.

leverages this information to perform priority processing for the UEs running a latency-sensitive app session.

5G Services Classes: 5G networks are aiming to support three broad classes of services, (i) enhanced Mobile Broad Band (eMBB), (ii) Ultra-Reliable and Low Latency Communication (URLLC), and (iii) delay-tolerant low-complexity Massive IoT (MIIoT) [6, 36, 37, 67, 97].

UE Classes at AMF: An AMF can differentiate between the low-complexity (MIIoT) and other (eMBB & URLLC) devices from the information provided by RAN or UE.⁹ An AF, on behalf of app [46], can inform which eMBB or URLLC device is to be treated normally or with high priority by the control plane. In the context of delay tolerance, our modified AMF can differentiate between the following classes of UEs:

- (1) *EdgeWarp-LP*: These are delay-tolerant low-complexity devices such as those used for smart cities, e.g., smart meters. A delay of up to 10 s is acceptable for these UEs [37, 67, 97]. *EdgeWarp*’s AMF is configured to have *lowest priority* for such UEs.
- (2) *EdgeWarp-MP*: Includes the rest of devices not included in *EdgeWarp-LP*, such as eMBB and URLLC. Such UEs are typically used in agriculture, education, media production, etc [37, 39]. Voice services can also be included in this class of UEs. These UEs have *medium priority* in *EdgeWarp*.
- (3) *EdgeWarp-HP*: These are devices from the *EdgeWarp-MP* having an active latency-sensitive data session with an edge app. These UEs have deadlines in the order of a few ms [36, 37]. Autonomous vehicles [69] and VR headsets [85] are key examples of such UEs. The server side of a latency-sensitive edge app adds a UE to this

⁹UE context at AMF (table 5.2.2.2.2-1 of the 3GPP document [2]) contains information if a UE is a low-complexity device such NB-IoT, LTE-M or RedCap (reduced capability device). Section 1.8 of the 3GPP document [1] suggests a UE can explicitly inform the core network that it is a low-priority device.

class when an eMBB or URLLC device starts a latency-sensitive data session with it. These UEs have *highest priority* in *EdgeWarp*.

Priority Processing for Latency-Sensitive Apps: We leverage the UE class information at the AMF to perform priority processing for latency-sensitive apps at RAN and AMF. Radio Resource Control (RRC) layer [6] provides a control plane signaling protocol between a UE and RAN. We maintain three separate priority queues for the RRC messages at the BS. The Next Generation Application Protocol (NGAP) provides the control plane signaling protocol between the RAN and AMF. We maintain three priority queues at the BS for the NGAP messages from the AMF. Similarly, we maintain three queues at the AMF from the control messages received from the BS. UEs at the RAN and AMF are identified with their unique IDs, *RAN UE NGAP ID* and *AMF UE NGAP ID*, respectively. The priority levels are configurable in our design and can be updated by network operators based on their preferences.¹⁰ If a high-priority queue is empty, we process messages from one level lower priority queue. Our design requires no changes at the UE. The following are sequence of steps involved when an edge app calls the APIs in Table 3 to prioritize their processing.

- (1) When a UE transitions from idle to connected mode [4], the AMF provides the required session state [6] to the BS. The AMF informs the BS whether a UE is a delay-tolerant low-end device (*EdgeWarp-LP*) or a high-end device (*EdgeWarp-MP* or *EdgeWarp-HP*).
- (2) The UE starts a data session with a latency-sensitive edge app hosted at the MEC platform. The edge app acts as an AF.
- (3) The edge app sets the highest priority for the UE with a call to *Set_Priority* API (in table 3) on AMF with the UE IP address as the UE’s unique ID. The AMF (having access to the complete UE’s control plane state) translates the UE’s IP address to the user unique IDs *RAN UE NGAP ID* and *AMF UE NGAP ID* at the BS and AMF, respectively. The AMF sets the highest priority for the UE on RAN by calling the *Set_Priority* API (table 3) exposed by RAN with *RAN UE NGAP ID* as a unique UE’s ID.
- (4) With a call to the above APIs, the BS and AMF add the UE to the high-priority list maintained for *EdgeWarp-HP*.
- (5) When the UE finishes the data session with the latency-sensitive edge app, the edge App reverts its priority to default with a call to *Revert_Priority* API (in Table 3) exposed by AMF. Consequently, the AMF calls *Revert_Priority* on RAN to revert the UE’s priority to default.

¹⁰The control processing threads at the BS and AMF by default process 6/10, 3/10, and 1/10 control messages from *EdgeWarp-HP*, *EdgeWarp-MP*, and *EdgeWarp-LP* queues, respectively.

5 Implementation

5.1 Target BS Prediction

We implement the LSTM network for target BS prediction in Python using *Keras version 2.9.0*. The network has 8 layers and 15,923 trainable parameters. Our training dataset combines radio network traces from high-speed trains [109], driving tests [27], and miscellaneous mobility scenarios [79], totaling 23630 handover events sampled at ≈ 50 ms intervals. The datasets include:

- (1) **High-speed trains data set:** This data set [109] was collected in high-speed trains (for a study published in ACM SIGCOMM [64]) where faster mobility can be a challenge for emerging edge applications. This data set has a total of 17085 handover events.
- (2) **Driving data set:** The data set [27] was collected during driving tests for a study published in IEEE INCP [63]. This data set has a total of 3947 handover events.¹¹
- (3) **Miscellaneous data set:** This data set includes multiple traces published to MobileInsight [79] website with no explicit mention of mobility scenarios. This data set has a total of 2598 handover events.

5.2 Modified External Data Store

We implement all our modifications (as discussed in §4.4) in the Redis [87] and Memcached [76] data stores. We implement our new APIs in Table 2 with approximately 1,392 lines of code changes in the open-source code [61] for Redis. For Memcached, we made our changes on top of its common client APIs (get, set, delete, etc.).

We used three versions of Redis clients, each one in C++ [96], Java [62], and Python [88]. We also implement a Python version of the modified Memcached data store on top of the *pymemcache 4.0.0* Python package. We interfaced our modified data stores with the following apps:

CarMap: We modify the open-source C++ code-base of CarMap [11, 47] to use the C++ version of our modified Redis data store for application state storage and updates. Our changes to CarMap comprise of ≈ 477 lines of code.

EMP: We modify the open-source Java code-base of EMP [94, 112] to use the Java version of our modified Redis data store. Our changes to EMP consist of ≈ 350 lines of code.

5.3 Edge App Simulation Framework

State properties across edge apps differ and it’s impractical to update many apps due to limited options. Therefore, we created a Python-based simulation framework for better experimental coverage. It allows users to configure their app

¹¹This data set is not used for training the LSTM network to avoid over-fitting.

state properties (i.e., state size, dynamic state percentage, state update rate) on the server side and generates app traffic on the client side. The framework interfaces with both modified Redis and Memcached data stores. The simulation framework consists of ≈ 1147 lines of code.

5.4 App-Aware Control Plane Processing

We have modified the open-source C/C++ code-base of a recently proposed cellular control plan [12, 80] to implement app-aware control plane processing (§4.5). We implement the RAN-related APIs of Table 3 in the control traffic generator (emulating both UE and BS) of [80]. Similarly, we implement the AMF-related APIs of Table 3 in the control plane function (AMF + SMF) of [80]. Our changes to [80] consist of the addition of ≈ 967 lines of code.

6 Evaluation

In this section, we show the following:

- (1) **Improvement in Downtime for Real Edge Apps:** *EdgeWarp* reduces app downtime during an app handover by up to 14 $\times$ and 15.4 $\times$ for CarMap and EMP, respectively.
- (2) **Impact of Proactive App State Migration:** *EdgeWarp* reduces blocking app state migration time by more than 2.5 $\times$ for both CarMap and EMP.
- (3) **Impact of App-Aware Control Plane Processing:** *EdgeWarp* reduces 5G handover completion time by up to 3 orders of magnitude for latency-sensitive apps.
- (4) **Comprehensive Apps Coverage:** Our sensitivity analysis using our simulation framework (§5.3) shows *EdgeWarp* can reduce application downtime for a wide variety of applications.
- (5) **Ablation Study:** We conduct a detailed ablation study to isolate and quantify the specific contribution of each key component, demonstrating the individual impact of our target BS prediction (§6.5.1), two-step state synchronization (§6.4), and app-aware control plane processing (§6.6).

6.1 Settings and Methodology

We evaluate *EdgeWarp* on a testbed comprising one client laptop and eight dual-socket servers, each hosting different components (Figure 7). Both source and destination sites consisted of a BS emulator, a cellular core network, edge applications, and a mobility handler with a target BS predictor module. The servers operate on an Intel Xeon(R) Gold 5220 CPU @ 2.20 GHz and have 128 GB memory, while the laptop has 20 GB RAM, 512 GB SSD, and a GeForce GTX 1060 6 GB GPU. To ensure statistical significance and capture variability, we repeated each experiment a minimum of 100 times for all plots displaying distributional data.

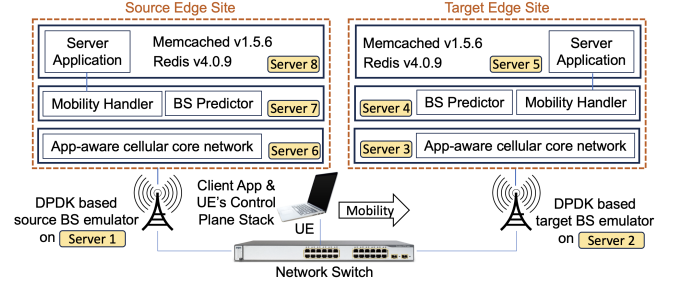


Figure 7: Testbed for experiments.

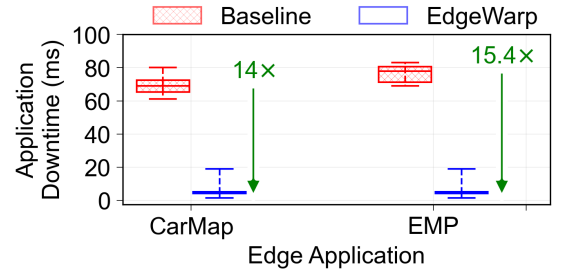


Figure 8: Overall improvement in app downtime during an app handover for real edge apps CarMap [11] and EMP [112].

6.2 Overall Improvement with Real Apps

In this section, we show improvement in app downtime with both proactive application state migration and app-aware control plane processing of *EdgeWarp*. Figure 8 shows app downtime during an app handover for CarMap [11] and EMP [112]. We use the default configuration for both CarMap and EMP.¹² In Figure 8, “Baseline” represents coordinated but reactive app handover (i.e., representing the coordinated handover as shown in Figure 1b) and “EdgeWarp” represents the modified application version with our proposed design (Figure 1d). Figure 8 shows 14 $\times$ and 15.4 $\times$ improvement in median app downtime for CarMap and EMP, respectively, when using *EdgeWarp*. In the following sections, we highlight the contribution of different modules to the overall improvement in app downtime.

6.3 Target BS Prediction with Real Traces

Figure 9 shows the earliest time to predict a target BS with the real 5G/4G radio network traces collected in different mobility scenarios. The figure shows 92.9%, 97.7% and 92.4% true predictions 100 ms before an actual handover with traces collected during driving [27], in high-speed trains [64], and miscellaneous mobility scenarios [79], respectively. With a 100 ms advanced mobility hint (i.e., predicting the target

¹²Row 1 of Table 4 and Table 5 shows default configuration for CarMap and EMP, respectively.

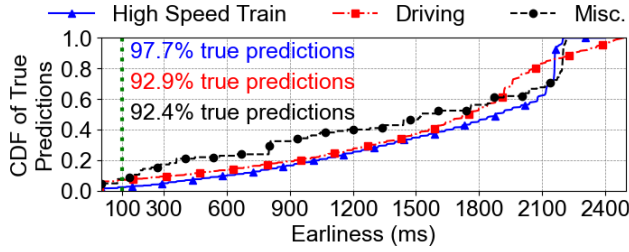


Figure 9: Accuracy of the early target BS prediction on three real data sets. The annotated statistics show the accuracy of predicting target BS 100 ms ahead of handover.

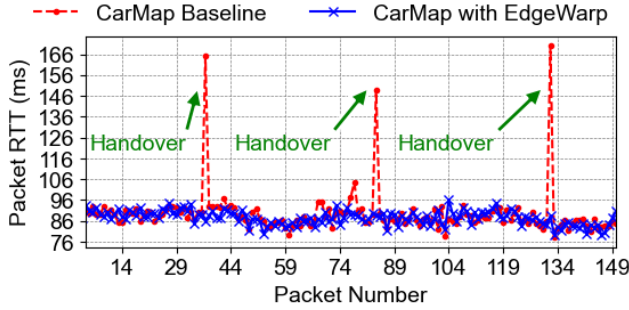


Figure 10: Improvement in message RTT during an app handover when using CarMap with *EdgeWarp* as compared to CarMap Baseline. Settings row 1 of table 4.

BS 100 ms before the handover), on average 97% true handover predictions are possible. The theoretical upper limit of true predictions is 99.3%, 99.5%, and 99% in high-speed trains, during driving, and miscellaneous mobility scenarios, respectively. The percentage is computed as the proportion of times when at least one radio sample is received for the true target BS before the handover.

On average in $\approx 3\%$ of cases, a correct prediction cannot be made 100 ms in advance. These instances are handled as follows. In $\approx 1.7\%$ of cases involving incorrect predictions, we first migrate to the wrong edge app instance and then perform a reactive migration to the optimal edge app instance once the client attaches to the optimal BS. This two-step process incurs no extra latency penalty—only a modest bandwidth overhead. In the remaining $\approx 1.3\%$ of cases, where the prediction is correct but arrives <100 ms before the event, *EdgeWarp* still outperforms the baseline (Figure 12).

A mobility hint received 100 ms in advance is enough for the edge apps we tested—both CarMap (a localization app) and EMP (a safety app) require less than 100 ms to provide less than 10 ms blocking state migration time with default configurations (row 1 of the table 4 & 5). Other safety and localization apps with similar state properties are also expected to perform similarly to CarMap and EMP.

App State Size (MBs)	Dynamic State (%)	Migration Time (ms)	
		Baseline	<i>EdgeWarp</i>
2.3	4.1	82.4	→ 6.8
4.3	47.1	134.2	→ 45.5
5.4	57.3	158.9	→ 62.8

Table 4: Reduction in CarMap’s average blocking state migration time with *EdgeWarp*’s modified Redis data store. We vary the frame upload interval at the vehicle to experiment with different state sizes.

We also evaluate the inference time for the BS prediction pipeline. The median inference times for a batch of 50 and 500 users are 8.7 ms and 9.5 ms, respectively.

6.4 Impact of Proactive State Migration on Real Applications

This section assesses the efficacy of *EdgeWarp*’s two-stage state synchronization in real edge apps using Redis store.

6.4.1 Improvement in CarMap with EdgeWarp. In Table 4, we compare the performance of CarMap during an app handover in terms of blocking application state migration time—the time from the start of a 5G handover to the completion of state migration. We compare two versions of CarMap: (i) CarMap Baseline represents existing systems where application state migration starts with a 5G handover and (ii) CarMap with *EdgeWarp* represents the case when the application state is stored in our modified Redis data store and two-step state synchronization is performed. Here the two-step state synchronization is performed with a mobility hint of 100 ms that results on average in 97% correct target BS predictions (Figure 9). Table 4 shows that with *EdgeWarp*, the application blocking state migration time drops to 6.8 ms and 62.8 ms from 82.3 ms and 158.9 ms when the application dynamic state varies from 4.1% to 57.3%, respectively. CarMap gets the benefits of proactive application state migration in 97% cases and performs like the baseline in 3% cases.

Figure 10 shows message RTT measured on the vehicle when three app handovers are performed during an experiment. The figure shows $\approx 2\times$ increase in RTT during an app handover when the vehicle client is served with CarMap Baseline. In contrast, CarMap with *EdgeWarp* does not see any significant change in message RTT during an app handover. This improvement is primarily attributed to the mobility hint-based two-step state synchronization provided in the modified Redis data store.

6.4.2 Improvement in EMP with EdgeWarp. In Table 5, we compare the performance of EMP baseline and EMP integrated with the modified Redis data store during an app

App State Size (MBs)	Dynamic State (%)	Migration Time (ms)	
		Baseline	EdgeWarp
0.46	100	77	→ 0.04
0.71	100	89	→ 28
0.95	100	145	→ 59

Table 5: Reduction in EMP’s average blocking state migration time when using *EdgeWarp*’s modified data store. We vary frames per vehicle at the edge server to experiment with different state sizes.

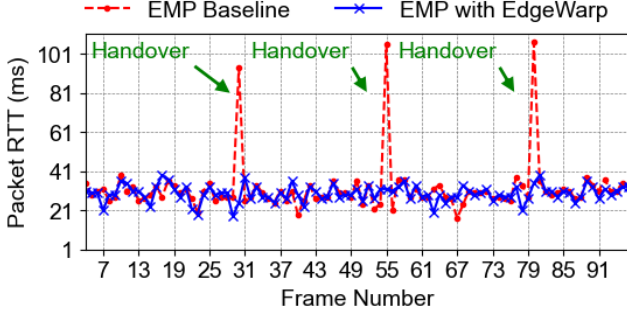


Figure 11: Improvement in message RTT during an app handover when using EMP with *EdgeWarp* as compared to EMP Baseline. Setting row 1 of table 5.

handover. The table shows that the blocking application state migration time drops to ≈ 0 ms when using *EdgeWarp*’s modified data store with app state size ≈ 0.46 MBs. EMP with *EdgeWarp* shows 3.2 $\times$ and 2.5 $\times$ improvement over the baseline with app state size 0.71 MBs and 0.95 MBs, respectively. The app state properties depend on the application logic. EMP has a relatively smaller state size as compared to CarMap but provides higher blocking state migration time because all the application state is dynamic.

Figure 11 shows message RTT measured on the vehicle when three app handovers are performed during an experiment. The figure shows more than 3 $\times$ improvement in EMP with *EdgeWarp* as compared to the baseline. These improvements are attributed to the mobility hint-based proactive application state synchronization provided by the *EdgeWarp*’s modified Redis data store.

6.5 State Migration Sensitivity Analysis

In this section, we show the impact of varying mobility hints on blocking state migration time for CarMap. As real apps provide limited options to experiment with different state properties, we also show the impact of varying dynamic state sizes with our simulation framework (§5.1).

6.5.1 Impact of mobility hint. The advanced mobility hint varies across handovers (as shown in Figure 9) due to changing radio signal strength. In Figure 12, we quantify the impact

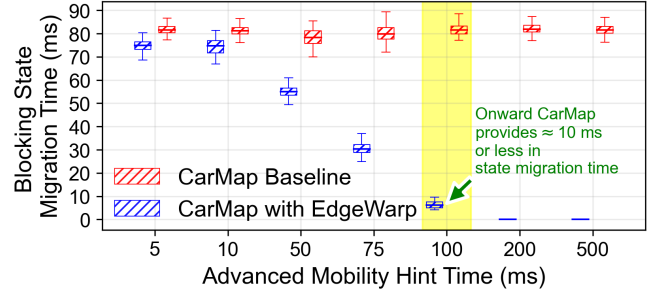


Figure 12: Impact of early mobility hint on CarMap’s blocking state migration time. Frame upload interval = 1 s. Delay budget ≈ 20 ms [15].

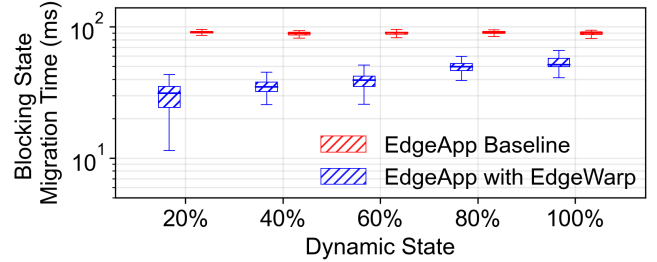


Figure 13: *EdgeWarp*’s application downtime with varying dynamic state size. The dynamic state update rate = 50 updates/s. The Y-axis is on a log scale.

of different advanced mobility hints on CarMap’s blocking state migration time. CarMap baseline performs very close to CarMap with *EdgeWarp* when the advanced mobility hint is under 10 ms. CarMap with *EdgeWarp* performs 1.5 $\times$ better than CarMap baseline in median blocking state migration time with an advanced mobility hint of 50 ms. When the advanced mobility hint is greater than 100 ms, CarMap with *EdgeWarp* provides a median blocking state migration time close to zero as compared to 80 ms for CarMap baseline. The figure shows an advanced mobility hint of 100 ms reduces CarMap blocking state migration time well below its network delay budget of 20 ms [11].

6.5.2 Impact of varying dynamic App state size. To understand *EdgeWarp*’s impact on a variety of apps, we simulate applications having different state properties with our simulation framework. The simulated apps store and perform proactive app state migration with our modified Memcached data store (§5.2). In Figure 13, we compare the blocking state migration time for the simulated app (baseline) and with *EdgeWarp*. We vary the app dynamic state from 20% to 100% with a total state size of 3 MBs (the observed dynamic state size for real edge apps, as shown in table 4 & 5). We set the update rate of every dynamic state object to 50 updates/s to simulate high-frequency updates [111]. With 20% dynamic state, *EdgeWarp* performs 2.9 $\times$ better in median blocking

Control Procedures/second	20K	25K	30K	35K
Without <i>EdgeWarp</i>	23%	30%	36%	42%
With <i>EdgeWarp</i>	0%	0%	0%	0%

Table 6: The % of missed deadlines for CarMap, during a handover by Existing 5G which drops to zero with *EdgeWarp*. Load represents control procedures per second. Lower % is better.

state migration time as compared to the baseline. The improvement of *EdgeWarp* over the baseline reduces as the dynamic state percentage increases. With a 100% dynamic state, the performance of *EdgeWarp* is still $1.7\times$ better as compared to the baseline. This is due to *EdgeWarp*'s capability of proactively migrating some or all the app state following advanced mobility hints.

6.6 App-Aware Control Plane Impact

In this section, we compare the performance of *EdgeWarp* with *Existing 5G*—a DPDK-based Open-Air Interface implementation of 5G control plane [13] where the AMF is not aware of the user app.

6.6.1 Impact on app performance. It is projected [74] that 5G IoT will constitute 60% of low-priority apps (*EdgeWarp-LP*), 28% for medium-priority apps (*EdgeWarp-MP*), and 12% for high-priority apps (*EdgeWarp-HP*). Latency-sensitive connected and autonomous vehicles, such as CarMap, have a network delay budget of ≈ 20 ms [15]. Table 6 illustrates the percentage of responses that missed deadlines¹³ for the CarMap app when served by *Existing 5G*. The table indicates a 19% increase in missed deadlines for CarMap as the load increases from 20K to 35K (a moderate load range [13, 81]) when served with *Existing 5G*. In contrast, when CarMap is served with *EdgeWarp*, the percentage of missed deadlines drops to zero across all load scenarios. The performance gain for CarMap in Table 6 is due to more than two orders of magnitude reduction in *handover* completion time when using *EdgeWarp* (details in §6.6.2).

6.6.2 Impact on handover completion time. Figure 14 shows *handover* completion time when the 5G control plane is serving a varying number of control procedures per second. The figure shows that the median *handover* completion time for high-priority latency-sensitive applications (*EdgeWarp-HP*) remains around 4.5 ms even when the load on the control plane increases from 20K to 35K control procedures per second. This is two orders of magnitude less than the case when *Existing 5G* serves the users. As a trade-off, we observe around $2.1\times$ increase in *handover* completion time for low priority delay tolerant applications (*EdgeWarp-LP*) as

¹³The missed app deadlines are solely attributed to the 5G handover. State migration is not a factor in this experiment.

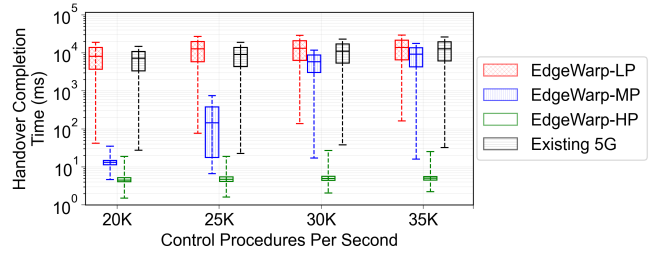


Figure 14: A comparison of *handover* completion time for *EdgeWarp* and *Existing 5G* with uniform control traffic. Traffic split 60% *EdgeWarp-LP*, 28% *EdgeWarp-MP*, and 12% for *EdgeWarp-HP*, as projected in [74].

compared to the *Existing 5G*. However, this is an acceptable trade-off as the *EdgeWarp-LP* can tolerate a delay of up to 10 s [37, 67, 97] and it is still < 1 s. In all load scenarios, we observe the medium priority delay tolerant applications (*EdgeWarp-MP*) has a lower *handover* completion time as compared to the *Existing 5G*, which suggests no degradation in the performance of *EdgeWarp-MP*.

7 Related Work

Stateful Edge Applications Migration: A recent work, Falcon [78] introduces a stream processing framework for stateful operator reconfiguration in edge-cloud environments, achieving minimal disruption through dual routing, marker-based synchronization, and emission filters. It supports reactive source mobility by detecting movement between edge nodes and migrating the state while maintaining processing continuity. *EdgeWarp* complements this work by enabling proactive migration through accurate target base station prediction and jointly optimizing network session and application migration, whereas Falcon focuses primarily on application state migration, which may introduce delays when network and application migrations are not synchronized.

Target BS Prediction: In prior work [22, 29, 98], target BS prediction is performed using support vector machines, handover history, and LTE event patterns. However, these studies focused on pre-5G networks, which differ from 5G with mmWave having limited range and more susceptible to physical obstacles [89]. Recent 5G-based approaches employed deep neural networks [60], historical handovers [8], and LSTM networks [86]. Notably, all these methods lack early prediction timings and provide relatively low prediction accuracy ($< 85\%$).

Resilience, Placement, and Load Balancing at the Edge: CESSNA [49] uses message logging and periodic state checkpointing for consistent recovery in edge failure and client mobility cases. Portkey [84] designs a key-value store adapting data placement per client mobility and data patterns, using passive profiling and active probing. EdgeBalance [110]

achieves load balancing in edge apps through CPU load prediction and an adaptive server load distribution. All these proposals can be used with our proposed design to further improve the edge apps' performance.

Mobility Impact on Application Performance: A recent work [50] quantifies the impact of mobility on app performance, power consumption, and signaling overheads. This work designs a *handover* prediction module *Prognos* and demonstrates its ability to improve real app performance. This work does not consider the problem of application session migration during mobility. In addition, this work does not speed up 5G handovers for latency-sensitive edge apps.

Reducing Cellular Control Plane Latency: Recent cellular control plane designs [13, 14, 55, 59, 66, 81] aim to reduce control plane delays. L25GC [55] uses shared-memory communication, smart buffering at UPF and a quick failure recovery protocol. CoreKube [59] leverages a stateless cellular core design for scalability and failure resilience. MMLite [81] enables procedure prioritization but lacks an interface for edge apps to dynamically influence control plane processing during a latency-sensitive data session. The highlighted designs can be integrated into our *EdgeWarp* to augment performance, particularly in terms of failure recovery.

8 Discussion

Improving Mobility Hint: Figure 9 shows 97% correct predictions on average with a 100 ms mobility hint. Prediction accuracy can be further improved when apps (e.g., autonomous vehicles) inform the mobility handler about motion intentions based on destination settings, and the operator considers historical handover data and user GPS coordinates.

Target BS Predictor Training: We train the target BS predictor module offline with diverse mobility traces. The offline model provides good performance. However, telecom operators can also train it online on a BS or edge site to learn new mobility trends for more accurate predictions.

Practical Considerations and Scalability: *EdgeWarp* is designed for applications with small-to-moderate state sizes, which are common in interactive edge computing. For instance, the CarMap [11] application has a total state of 2.3 MB, of which our protocol proactively migrates only the low-update-rate portion of ≈ 460 KB (20% of the total user state). This proactive transfer represents a minimal bandwidth overhead that is easily accommodated by typical 5G edge network bandwidths (>100 Mbps). For applications with very large state sizes (e.g., >100 MB), the latency of reading from and writing to an external data store can become a bottleneck, and the benefits of our approach may be limited. Finally, performing state synchronization on a per-user basis allows

our approach to scale efficiently with the number of concurrent users, as the migration overhead for one user does not impact the performance of others.

Always Sync Approach: Data stores, such as Redis [87] and ZooKeeper [54] allow state replication on the backup replicas for fault tolerance. However, always replicating the user state on the backup (i) can have a significant bandwidth overhead due to the typically high update rates of the latency-sensitive apps [11, 85, 111, 112], and (ii) there may be less benefit of proactive replication due to the state soon becoming outdated due to frequent updates.

Backward Compatibility and Operational Trade-offs: *EdgeWarp*'s design aligns with existing standards to ensure a clear deployment path. Our modifications to the AMF are a logical extension of current UE classification mechanisms specified in 3GPP TS 23.501 [6], and our new AF APIs are consistent with the ETSI GS MEC 021 [43, 46] framework for app mobility. *EdgeWarp*'s approach requires no changes at the UE, simplifying adoption. Regarding operational trade-offs, while our app-aware control plane increases low-priority (LP) flow latency by approximately $2.1\times$ under load, the absolute completion time remains under 1 second, well within the acceptable delay budget of up to 10 seconds for these applications [37, 67, 97]. Furthermore, the priority weights are operator-configurable, allowing for flexible policy enforcement to meet different fairness requirements and SLAs.

9 Concluding Remarks

In this work, we addressed the problem of enabling stateful mobile edge applications. We identified the key architectural issues in enabling such demanding applications. We posit that there is a need to rethink coordination between edge apps and 5G networks. To this end, we proposed a new cross-layer system architecture, *EdgeWarp*. Our evaluations with real edge apps and simulations show that *EdgeWarp* takes a major step towards enabling such apps by significantly reducing app downtime during mobility. We hope our work will ignite an important discourse on enabling emerging mobile applications over 5G and beyond.

10 Acknowledgments

We thank our shepherd Stephen Lee and the anonymous ACM/IEEE Symposium on Edge Computing (SEC) reviewers for their valuable feedback.

References

- [1] 3GPP. "ETSI TS 124 008 V15.9.0 (2020-10)". https://www.etsi.org/deliver/etsi_ts/124000_124099/124008/15.09.00_60/ts_124008v150900p.pdf. [Online; accessed 19-June-2024].
- [2] 3GPP. Procedures for the 5G System (5GS); Stage 2 (Release 16). https://www.3gpp.org/ftp/Specs/archive/23_series/23.502/, 2018. [Online;

- [1] accessed 18-June-2024].
- [3] 3GPP. ETSI TS 123 502 V15.5.1 (2019-05). https://www.etsi.org/deliver/etsi_ts/123500_123599/123502/15.05.01_60/ts_123502v150501p.pdf, 2023. [Online; accessed 21-June-2023].
- [4] 3GPP Ref #: 124.301. Non-Access-Stratum (NAS) protocol Stage 3. www.3gpp.org/dynareport/24301.htm, 2023.
- [5] 3GPP Ref #: 23.401. *General Packet Radio Service (GPRS) enhancements for Evolved Universal Terrestrial Radio Access Network (E-UTRAN) access*, 2016.
- [6] 3GPP Ref #: 23.501. System architecture for the 5G System (5GS). <https://portal.3gpp.org/desktopmodules/Specifications/SpecificationDetails.aspx?specificationId=3144>, 2023.
- [7] 3GPP Ref #: 29.272. *Mobility Management Entity (MME) and Serving GPRS Support Node (SGSN) related interfaces based on Diameter protocol*, 2016.
- [8] Abdah, Hadeel and Barraca, João Paulo and Aguiar, Rui L. Handover Prediction Integrated with Service Migration in 5G Systems. In *ICC 2020 - 2020 IEEE International Conference on Communications (ICC)*, pages 1–7, 2020.
- [9] Abouaoumar, Amine and Mlika, Zoubeir and Filali, Abderrahime and Cherkaoui, Soumaya and Kobbane, Abdellatif. A deep reinforcement learning approach for service migration in mec-enabled vehicular networks. In *2021 IEEE 46th conference on local computer networks (LCN)*, pages 273–280. IEEE, 2021.
- [10] Addad, Rami Akrem and Dutra, Diego Leonel Cadette and Bagaa, Miloud and Taleb, Tarik and Flinck, Hannu. Fast Service Migration in 5G Trends and Scenarios. *IEEE Network*, 34(2):92–98, 2020.
- [11] Fawad Ahmad, Hang Qiu, Ray Eells, Fan Bai, and Ramesh Govindan. CarMap: Fast 3D Feature Map Updates for Automobiles. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pages 1063–1081, 2020.
- [12] Mukhtiar Ahmad, Syed Muhammad Nawazish Ali, Muhammad Taimoor Tariq, Syed Usman Jafri, Adnan Abbas, Syeda Mashal Abbas Zaidi, Muhammad Basit Iqbal Awan, Zartash Afzal Uzmi, and Zafar Ayyub Qazi. Neutrino: A fast and consistent edge-based cellular control plane. *IEEE/ACM Transactions on Networking*, pages 1–16, 2022.
- [13] Mukhtiar Ahmad, Syed Usman Jafri, Azam Ikram, Wasiq Noor Ahmad Qasmi, Muhammad Ali Nawazish, Zartash Afzal Uzmi, and Zafar Ayyub Qazi. A low latency and consistent cellular control plane. In *Proceedings of the Annual conference of the ACM Special Interest Group on Data Communication on the applications, technologies, architectures, and protocols for computer communication*, pages 648–661, 2020.
- [14] Mukhtiar Ahmad, Muhammad Ali Nawazish, Muhammad Taimoor Tariq, Muhammad Basit Iqbal Awan, Muhammad Taqi Raza, and Zafar Ayyub Qazi. Enabling emerging edge applications through a 5g control plane intervention. In *Proceedings of the 18th International Conference on emerging Networking EXperiments and Technologies*, pages 386–400, 2022.
- [15] Ahmad, Fawad and Shin, Christina and Pang, Weiwu and Cashman, Jacob and Leong, Branden and Govindan, Ramesh. iDriving: Toward Safe and Efficient Infrastructure-directed Autonomous Driving. *arXiv preprint arXiv:2207.08930*, 2022.
- [16] Akdeniz, Mustafa Riza and Liu, Yuanpeng and Samimi, Mathew K and Sun, Shu and Rangan, Sundeeep and Rappaport, Theodore S and Erkip, Elza. Millimeter wave channel modeling and cellular capacity evaluation. *IEEE journal on selected areas in communications*, 32(6):1164–1179, 2014.
- [17] American Tower. EDGE DATA CENTERS. <https://www.americantower.com/us/solutions/data-centers/edge/>. [Online; accessed 04-Dec-2023].
- [18] Ganesh Ananthanarayanan, Paramvir Bahl, Peter Bodik, Krishna Chintalapudi, Matthai Philipose, Lenin Ravindranath, and Sudipta Sinha. Real-time video analytics: The killer app for edge computing. *Computer*, 50(10):58–67, 2017.
- [19] Anne Geelen. Deutsche Telekom, Ericsson and Samsung demonstrate 5G E2E Network Slicing on a commercial 5G device. https://www.telekom.com/en/media/media-information/archive/5g-e2e-network-slicing-630550?sharingsservice=linkedin_share%22%20\T1\|%20%22.YNmTtaWT4A8.linkedin. [Online; accessed 17-May-2024].
- [20] AWS. 5G and Edge Computing Enhances Connected and Autonomous Experiences. <https://d1.awsstatic.com/autonomous-connected.pdf>. [Online; accessed 13-June-2024].
- [21] AWS. Cloud Gaming at the Edge. <https://d1.awsstatic.com/product-marketing/Outposts/AWS%20Gaming%20Infographic.pdf>. [Online; accessed 17-June-2024].
- [22] Zdenek Becvar. Efficiency of Handover Prediction Based on Handover History. *J. Convergence Inf. Technol.*, 4(4):41–47, 2009.
- [23] Ali J Ben Ali, Marziye Kouroshli, Sofiya Semenova, Zakieh Sadat Hashemifar, Steven Y Ko, and Karthik Dantu. Edge-slam: Edge-assisted visual simultaneous localization and mapping. *ACM Transactions on Embedded Computing Systems*, 22(1):1–31, 2022.
- [24] Anushka Biswas and Hwang-Cheng Wang. Autonomous vehicles enabled by the integration of iot, edge intelligence, 5g, and blockchain. *Sensors*, 23(4):1963, 2023.
- [25] Mathieu Bouet and Vania Conan. Mobile Edge Computing Resources Optimization: A Geo-Clustering Approach. *IEEE Transactions on Network and Service Management*, 15(2):787–796, 2018.
- [26] Tuan Cao, Marcos Vaz Salles, Benjamin Sowell, Yao Yue, Alan Demers, Johannes Gehrke, and Walker White. Fast checkpoint recovery algorithms for frequently consistent applications. In *Proceedings of the 2011 ACM SIGMOD International Conference on Management of data*, pages 265–276, 2011.
- [27] cathyli93. Rperf Dataset Release (ICNP’21). <https://github.com/cathyli93/icnp21-reconfig-dataset>. [Online; accessed 14-Jan-2023].
- [28] Angelo Centonza and Henrik Basilier. Applied network slicing scenarios in 5G. <https://www.ericsson.com/en/reports-and-papers/ericsson-technology-review/articles/applied-network-slicing-scenarios-in-5g>. [Online; accessed 13-June-2024].
- [29] Chen, Xu and Mériaux, François and Valentin, Stefan. Predicting a user’s next cell with supervised learning based on channel states. In *2013 IEEE 14th workshop on signal processing advances in wireless communications (SPAWC)*, pages 36–40. IEEE, 2013.
- [30] Christian Kim. 5G and Massive IoT: legacy technologies will bridge the gap for now. <https://omdia.tech.informa.com/OM004695/5G-and-Massive-IoT-legacy-technologies-will-bridge-the-gap-for-now>. [Online; accessed 15-Feb-2023].
- [31] Christopher Clark and Keir Fraser and Steven Hand and Jacob Gorm Hansen and Eric Jul and Christian Limpach and Ian Pratt and Andrew Warfield. Live Migration of Virtual Machines. In *2nd Symposium on Networked Systems Design & Implementation (NSDI 05)*, Boston, MA, May 2005. USENIX Association.
- [32] CRIU. A project to implement checkpoint/restore functionality for Linux. <https://github.com/checkpoint-restore/criu>. [Online; accessed 17-June-2024].
- [33] Data Center Knowledge. Why Google Cloud and AT&T May Merge their Telco Edges, 2021.
- [34] Aditya Dhakal, Xukan Ran, Yunshu Wang, Jiasi Chen, and KK Ramakrishnan. Slam-share: visual simultaneous localization and mapping for real-time multi-user augmented reality. In *Proceedings of the 18th International Conference on emerging Networking EXperiments and Technologies*, pages 293–306, 2022.

- [35] Elbamby, Mohammed S and Perfecto, Cristina and Bennis, Mehdi and Doppler, Klaus. Toward low-latency and ultra-reliable virtual reality. *IEEE Network*, 32(2):78–84, 2018.
- [36] Ericsson. Cellular IoT Evolution for Industry Digitization. <https://www.ericsson.com/en/reports-and-papers/white-papers/cellular-iot-evolution-for-industry-digitalization>. [Online; accessed 17-June-2024].
- [37] Ericsson. Cellular IoT in the 5G era. <https://www.ericsson.com/4adb7e/assets/local/reports-papers/mobility-report/documents/2024/ericsson-mobility-report-november-2024.pdf>. [Online; accessed 21-June-2025].
- [38] ERICSSON. The essential building blocks of E2E network slicing. <https://foryou.ericsson.com/eso-network-slicing-e2e-paper.html>. [Online; accessed 17-June-2024].
- [39] Ericsson. Cellular iot evolution for industry digitalization, 2020.
- [40] Ericsson and Audi. 5G URLLC - Achieving industrial automation. <https://www.ericsson.com/en/cases/2020/accelerate-factory-automation-with-5g-urllc>.
- [41] ETSI. MEC Deployments in 4G and Evolution Towards 5G. https://www.etsi.org/images/files/etsiwhitepapers/etsi_wp24_mec_deployment_in_4g_5g_final.pdf. [Online; accessed 14-Jan-2023].
- [42] ETSI. Mobile Edge Computing (MEC); Radio Network Information API. https://www.etsi.org/deliver/etsi_gs/MEC/001_099/012/02.02.01_60/gs_MEC012v020201p.pdf. [Online; accessed 10-Jan-2023].
- [43] ETSI. Multi-access Edge Computing (MEC); Application Mobility Service API. https://www.etsi.org/deliver/etsi_gs/MEC/001_099/021/02.01.01_60/gs_mec021v020101p.pdf. [Online; accessed 06-June-2024].
- [44] ETSI. GS MEC 002: Multi-access Edge Computing (MEC); Phase 2: Use Cases and Requirements. https://www.etsi.org/deliver/etsi_gs/mec/001_099/002/02.01.01_60/gs_mec002v020101p.pdf, 2018.
- [45] ETSI. GS MEC 003: Multi-access Edge Computing (MEC); Framework and Reference Architecture. https://www.etsi.org/deliver/etsi_gs/MEC/001_099/003/02.02.01_60/gs_MEC003v020201p.pdf, 2018.
- [46] ETSI. MEC in 5G networks. https://www.etsi.org/images/files/ETSIWhitePapers/etsi_wp28_mec_in_5G_FINAL.pdf, 2018.
- [47] Fawad Ahmad. CarMap. <https://github.com/USC-NSL/CarMap>. [Online; accessed 09-Nov-2024].
- [48] Keerthana Govindaraj and Alexander Artemenko. Container live migration for latency critical industrial applications on edge computing. In *2018 IEEE 23rd International Conference on Emerging Technologies and Factory Automation (ETFA)*, volume 1, pages 83–90. IEEE, 2018.
- [49] Yotam Harchol, Aisha Mushtaq, James McCauley, Aurojit Panda, and Scott Shenker. CESSNA: Resilient edge-computing. In *Proceedings of the 2018 Workshop on Mobile Edge Communications*, pages 1–6, 2018.
- [50] Ahmad Hassan, Arvind Narayanan, Anlan Zhang, Wei Ye, Ruiyang Zhu, Shuwei Jin, Jason Carpenter, Z Morley Mao, Feng Qian, and Zhi-Li Zhang. Vivisecting mobility management in 5g cellular networks. In *Proceedings of the ACM SIGCOMM 2022 Conference*, pages 86–100, 2022.
- [51] He, Yuze and Ma, Li and Jiang, Zhehao and Tang, Yi and Xing, Guoliang. VI-eye: semantic-based 3D point cloud registration for infrastructure-assisted autonomous driving. In *Proceedings of the 27th Annual International Conference on Mobile Computing and Networking*, pages 573–586, 2021.
- [52] Motoshi Horii, Yuji Kojima, and Kenichi Fukuda. Stateful process migration for edge computing applications. In *2018 IEEE Wireless Communications and Networking Conference (WCNC)*, pages 1–6. IEEE, 2018.
- [53] Huawei Lab. Cloud VR Network; Solution White Paper. https://www.huawei.com/minisite/pdf/ilab/cloud_vr_network_solution_white_paper_en.pdf. [Online; accessed 17-June-2024].
- [54] Hunt, Patrick and Konar, Mahadev and Junqueira, Flavio Paiva and Reed, Benjamin. ZooKeeper: Wait-free Coordination for Internet-scale Systems. In *USENIX annual technical conference*, 2010.
- [55] Vivek Jain, Hao-Tse Chu, Shixiong Qi, Chia-An Lee, Hung-Cheng Chang, Cheng-Ying Hsieh, KK Ramakrishnan, and Jyh-Cheng Chen. L25gc: a low latency 5g core network based on high-performance nfv platforms. In *Proceedings of the ACM SIGCOMM 2022 Conference*, pages 143–157, 2022.
- [56] Ju1ce. April-Tag-VR-FullBody-Tracker. <https://github.com/ju1ce/April-Tag-VR-FullBody-Tracker>. [Online; accessed 17-June-2024].
- [57] Keras. LXD. <https://keras.io/>. [Online; accessed 14-Feb-2023].
- [58] Kundu, Ripan Kumar and Rahman, Akhlaqur and Paul, Shuva. A study on sensor system latency in vr motion sickness. *Journal of Sensor and Actuator Networks*, 10(3):53, 2021.
- [59] Larrea, Jon and Ferguson, Andrew E and Marina, Mahesh K. CoreKube: An Efficient, Autoscaling and Resilient Mobile Core System. In *Proceedings of the 29th Annual International Conference on Mobile Computing and Networking*, pages 1–15, 2023.
- [60] Lee, Changsung and Cho, Hyoungjun and Song, Sooeun and Chung, Jong-Moon. Prediction-based conditional handover for 5G mm-Wave networks: A deep-learning approach. *IEEE Vehicular Technology Magazine*, 15(1):54–62, 2020.
- [61] lettuce.io. Lettuce - Advanced Java Redis client. <https://github.com/redis/redis>. [Online; accessed 26-Aug-].
- [62] lettuce.io. Lettuce - Advanced Java Redis client. https://github.com/cpp-redis/cpp_redis. [Online; accessed 09-Nov-2024].
- [63] Qianru Li and Chunyi Peng. Reconfiguring cell selection in 4g/5g networks. In *2021 IEEE 29th International Conference on Network Protocols (ICNP)*, pages 1–11. IEEE, 2021.
- [64] Yuanjie Li, Qianru Li, Zhehui Zhang, Ghufan Baig, Lili Qiu, and Songwu Lu. Beyond 5g: Reliable extreme mobility management. In *Proceedings of the Annual conference of the ACM Special Interest Group on Data Communication on the applications, technologies, architectures, and protocols for computer communication*, pages 344–358, 2020.
- [65] Li, Jingya and Nagalapuri, Keerthi Kumar and Stare, Erik and Dwivedi, Satyam and Ashraf, Shehzad Ali and Eriksson, Per-Erik and Engström, Ulrika and Lee, Woonghee and Lohmar, Thorsten. 5G New Radio for Public Safety Mission Critical Communications. *arXiv preprint arXiv:2103.02434*, 2021.
- [66] Li, Yuanjie and Yuan, Zengwen and Peng, Chunyi. A control-plane perspective on reducing data access latency in lte networks. In *Proceedings of the 23rd Annual International Conference on Mobile Computing and Networking*, pages 56–69, 2017.
- [67] Olof Liberg. LTE-M and NB-IoT meet the 5G performance requirements. <https://www.ericsson.com/en/blog/2018/12/lte-m-and-nb-iot-meet-the-5g-performance-requirements>. [Online; accessed 08-May-2024].
- [68] Limelight. EDGE COMPUTE FOR GAMING. <https://www.limelight.com/resources/data-sheet/edge-compute-for-gaming/>. [Online; accessed 17-June-2024].
- [69] Lin, Shih-Chieh and Zhang, Yunqi and Hsu, Chang-Hong and Skach, Matt and Haque, Md E and Tang, Lingjia and Mars, Jason. The architectural implications of autonomous driving: Constraints and acceleration. In *Proceedings of the Twenty-Third International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 751–766, 2018.
- [70] Madhusanka Liyanage, Pawani Porambage, Aaron Yi Ding, and Anshuman Kalla. Driving forces for multi-access edge computing (mec)

iot integration in 5g. *ICT Express*, 7(2):127–137, 2021.

- [71] Wei Lu, Xianyu Meng, and Guanfei Guo. Fast service migration method based on virtual machine technology for mec. *IEEE Internet of Things Journal*, 6(3):4344–4354, 2018.
- [72] Lele Ma, Shanhe Yi, and Qun Li. Efficient service handoff across edge servers via docker container migration. In *Proceedings of the Second ACM/IEEE Symposium on Edge Computing*, pages 1–13, 2017.
- [73] Machen, Andrew and Wang, Shiqiang and Leung, Kin K. and Ko, Bong Jun and Salonidis, Theodoros. Live Service Migration in Mobile Edge Clouds. *IEEE Wireless Communications*, 25(1):140–147, 2018.
- [74] McKinsey & Compnay. The 5G era. <https://www.mckinsey.com/~media/mckinsey/industries/advanced%20electronics/our%20insights/the%205g%20era%20new%20horizons%20for%20advanced%20electronics%20and%20industrial%20companies/the-5g-era-new-horizons-for-advanced-electronics-and-industrial-companies.pdf>. [Online; accessed 18-June-2024].
- [75] MEC. MEC - Multi-access Edge Computing. <https://forge.etsi.org/rep/mec>. [Online; accessed 18-June-2024].
- [76] Memcached. Memcached. <https://memcached.org/>. [Online; accessed 07-May-2024].
- [77] Microsoft. About Edge Zone Preview. <https://docs.microsoft.com/en-us/azure/networking/edge-zones-overview>. [Online; accessed 18-June-2024].
- [78] Prithish Mishra, Nelson Bore, Brian Ramprasad, Myles Thiessen, Moshe Gabel, Alexandre Da Silva Veith, Oana Balmau, and Eyale De Lara. Falcon: Live reconfiguration for stateful stream processing on the edge. In *2024 IEEE/ACM Symposium on Edge Computing (SEC)*, pages 234–248. IEEE, 2024.
- [79] MobileInsight. MobileInsight. <http://www.mobileinsight.net/index.html>. [Online; accessed 18-June-2024].
- [80] Mukhtiar et al. Neutrino. <https://github.com/nsgLUMS/neutrino>. [Online; accessed 09-Nov-2024].
- [81] Vasudevan Nagendra, Arani Bhattacharya, Anshul Gandhi, and Samir R. Das. Mmlite: A scalable and resource efficient control plane for next generation cellular packet core. In *Proceedings of the 2019 ACM Symposium on SDN Research, SOSR '19*, 2019.
- [82] Narayanan, Arvind and Ramadan, Eman and Carpenter, Jason and Liu, Qingxu and Liu, Yu and Qian, Feng and Zhang, Zhi-Li. A first look at commercial 5G performance on smartphones. In *Proceedings of The Web Conference 2020*, pages 894–905, 2020.
- [83] Ngo, Mao V and Luo, Tie and Hoang, Hieu T and Ouek, Tony QS. Coordinated container migration and base station handover in mobile edge computing. In *GLOBECOM 2020-2020 IEEE Global Communications Conference*, pages 1–6. IEEE, 2020.
- [84] Joseph Noor, Mani Srivastava, and Ravi Netravali. Portkey: Adaptive key-value placement over dynamic edge networks. In *Proceedings of the ACM Symposium on Cloud Computing*, pages 197–213, 2021.
- [85] China Academy of Information and Communications Technology. *Virtual Reality/Augmented Reality White Paper*, 2018.
- [86] Ozturk, Metin and Gogate, Mandar and Onireti, Oluwakayode and Adeel, Ahsan and Hussain, Amir and Imran, Muhammad A. A novel deep learning driven, low-cost mobility prediction approach for 5G cellular networks: The case of the Control/Data Separation Architecture (CDSA). *Neurocomputing*, 358:479–489, 2019.
- [87] Redis. The open source, in-memory data store used by millions of developers as a database, cache, streaming engine, and message broker. <https://redis.io/>. [Online; accessed 07-May-2024].
- [88] Redis Inc. Redis Python Client. <https://github.com/redis/redis-py>. [Online; accessed 09-Nov-2024].
- [89] Roh, Wonil and Seol, Ji-Yun and Park, Jeongho and Lee, Byunghwan and Lee, Jaekon and Kim, Yungsoo and Cho, Jaeweon and Cheun, Kyungwhoon and Aryanfar, Farshid. Millimeter-wave beamforming as an enabling technology for 5G cellular communications: theoretical feasibility and prototype results. *IEEE Communications Magazine*, 52(2):106–113, 2014.
- [90] Satyanarayanan and Mahadev. The emergence of edge computing. *Computer*, 50(1):30–39, 2017.
- [91] ScreenBeam. How to Reduce Latency or Lag in Gaming. <https://www.screenbeam.com/wifihelp/wifi booster/how-to-reduce-latency-or-lag-in-gaming-2/>. [Online; accessed 18-June-2024].
- [92] SDxCentral. Amazon wavelength, 2021.
- [93] ShareTechNote. PDU Session Establishment in a Nutshell. https://www.sharetechnote.com/html/5G/5G_PDU Session Establishment.html, 2023.
- [94] Shawnxm. EMP. <https://github.com/Shawnxm/EMP>. [Online; accessed 09-Nov-2024].
- [95] Shi, Shu and Gupta, Varun and Hwang, Michael and Jana, Rittwik. Mobile VR on edge cloud: a latency-driven design. In *Proceedings of the 10th ACM multimedia systems conference*, pages 222–231, 2019.
- [96] Simon Ninon. C++11 Lightweight Redis client. https://github.com/cpp-redis/cpp_redis. [Online; accessed 09-Nov-2024].
- [97] Specification #: 38.913. *Study on Scenarios and Requirements for Next Generation Access Technologies; (Release 17)*.
- [98] Tan, Zhaowei and Li, Yuanjie and Li, Qianru and Zhang, Zhehui and Li, Zhehan and Lu, Songwu. Supporting mobile VR in LTE networks: How close are we? *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, 2(1):1–31, 2018.
- [99] Animesh Trivedi, Lin Wang, Henri Bal, and Alexandru Iosup. Sharing and caring of data at the edge. In *3rd {USENIX} Workshop on Hot Topics in Edge Computing (HotEdge 20)*, 2020.
- [100] Maarten van Steen and Andrew S. Tanenbaum. *Distributed Systems: Principles and Paradigms (3rd edition)*. Maarten van Steen, 2017.
- [101] Jiadai Wang, Jiajia Liu, and Nei Kato. Networking and communications in autonomous driving: A survey. *IEEE Communications Surveys & Tutorials*, 21(2):1243–1274, 2018.
- [102] Wang, Guozhang and Salles, Marcos Vaz and Sowell, Benjamin and Wang, Xun and Cao, Tuan and Demers, Alan and Gehrke, Johannes and White, Walker. Behavioral simulations in mapreduce. *arXiv preprint arXiv:1005.3773*, 2010.
- [103] White, Walker and Demers, Alan and Koch, Christoph and Gehrke, Johannes and Rajagopalan, Rajmohan. Scaling games to epic proportions. In *Proceedings of the 2007 ACM SIGMOD international conference on Management of data*, pages 31–42, 2007.
- [104] Hang Wu, Lixing Chen, Cong Shen, Wujie Wen, and Jie Xu. Online Geographical Load Balancing for Energy-Harvesting Mobile Edge Computing. In *2018 IEEE International Conference on Communications (ICC)*, pages 1–6, 2018.
- [105] Jingao Xu, Hao Cao, Zheng Yang, Longfei Shangguan, Jialin Zhang, Xiaowu He, and Yunhao Liu. {SwarmMap}: Scaling up real-time collaborative visual {SLAM} at the edge. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*, pages 977–993, 2022.
- [106] Xu, Dongzhu and Zhou, Anfu and Zhang, Xinyu and Wang, Guixian and Liu, Xi and An, Congkai and Shi, Yiming and Liu, Liang and Ma, Huadong. Understanding operational 5G: A first measurement study on its coverage, performance and energy consumption. In *Proceedings of the Annual conference of the ACM Special Interest Group on Data Communication on the applications, technologies, architectures, and protocols for computer communication*, pages 479–494, 2020.
- [107] Xu, Jingao and Cao, Hao and Yang, Zheng and Shangguan, Longfei and Zhang, Jialin and He, Xiaowu and Liu, Yunhao. {SwarmMap}: Scaling Up Real-time Collaborative Visual {SLAM} at the Edge. In

19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22), pages 977–993, 2022.

- [108] Yang, Run and He, Hui and Zhang, Weizhe. Multitier Service Migration Framework Based on Mobility Prediction in Mobile Edge Computing. *Wireless Communications and Mobile Computing*, 2021, 2021.
- [109] Yuanjie Li, Qianru Li, Zhehui Zhang, Ghufra Baig, Lili Qiu, Songwu Lu. REM-SIGCOMM20. https://www.dropbox.com/sh/g18byqo8idjkn3k/AABWSzJP_ayBMZAQSk0WdJdBa?dl=0. [Online; accessed 14-Jan-2023].
- [110] Zhang, Wei and Sharma, Abhigyan and Wood, Timothy. {EdgeBalance}:{Model-Based} Load Balancing for Network Edge Data Planes. In *3rd USENIX Workshop on Hot Topics in Edge Computing (HotEdge 20)*, 2020.
- [111] Zhang, Wuyang and Chen, Jiachen and Zhang, Yanyong and Raychaudhuri, Dipankar. Towards efficient edge cloud augmentation for virtual reality MMOGs. In *Proceedings of the Second ACM/IEEE Symposium on Edge Computing*, pages 1–14, 2017.
- [112] Zhang, Xumiao and Zhang, Anlan and Sun, Jiachen and Zhu, Xiao and Guo, Y Ethan and Qian, Feng and Mao, Z Morley. EMP: Edge-assisted multi-vehicle perception. In *Proceedings of the 27th Annual International Conference on Mobile Computing and Networking*, pages 545–558, 2021.
- [113] Zhe Zhou, Xintong Li, Xiaoyang Wang, Zheng Liang, Guangyu Sun, and Guojie Luo. Hardware-assisted service live migration in resource-limited edge computing systems. In *2020 57th ACM/IEEE Design Automation Conference (DAC)*, pages 1–6. IEEE, 2020.
- [114] Zhou, Pengyuan and Finley, Benjamin and Li, Xuebing and Tarkoma, Sasu and Kangasharju, Jussi and Ammar, Mostafa and Hui, Pan. 5G MEC computation handoff for mobile augmented reality. *arXiv preprint arXiv:2101.00256*, 2021.